

Instituto Politécnico Nacional

Escuela Superior de cómputo

Fundamentos de programación

Práctica 02 Variables y operadores en C

Una forma de hacer que el lenguaje C genere números aleatorios es emplear dos librerías aparte de la librería estándar de entrada y salida que es `stdio.h`, y ellas son `stdlib.h` y `time.h`, la primera para emplear a las funciones `rand()` y `srand()` y la segunda para emplear a la función `time()`. Para generar una semilla para crear números pseudo aleatorios emplee la siguiente línea

```
srand(time(NULL));
```

Para generar los números pseudo aleatorios necesitará una variable que limite el número máximo que puede generar aleatorios con la función `rand` de la siguiente forma

```
int numero;  
int limite = 100;  
numero = rand() % limite;
```

Del segmento de código anterior usted generará números aleatorios entre 0 y 100, veamos ahora el programa, introduzcalo en un editor, sávelo, compílelo y ejecútelo al menos diez veces, diga si en algún momento se sus ejecuciones el programa genero los mismos números.

```
1  #include <stdio.h>  
2  // las dos librerías necesarias  
3  #include <stdlib.h>  
4  #include <time.h>  
5  
6  int main(int argc, char *argv[]) {  
7      int numeroUno, numeroDos;  
8      int limite = 100;  
9      // Crea la semilla de los números aleatorios  
10     srand(time(NULL));  
11     // Genera dos números aleatorios  
12     numeroUno = rand() % limite;  
13     numeroDos = rand() % limite;  
14     // imprime los dos números aleatorios  
15     printf("numeroUno = %d\n", numeroUno);  
16     printf("numeroDos = %d\n", numeroDos);  
17     return 0;  
18 }  
19
```

Figura 1 Programa que genera dos números enteros aleatorios (aleatorioentero.c).

Ahora vamos a generar números aleatorios en coma flotante, agregamos algunas lineas de codigo más y obtenemos el programa de la figura 2 (aleatorioflotante.c).

```

1  #include <stdio.h>
2  // las dos librerias necesarias
3  #include <stdlib.h>
4  #include <time.h>
5
6  int main(int argc, char *argv[]) {
7      float numeroFloatUno, numeroFloatDos;
8      int numeroUno, numeroDos;
9      int numeroTres, numeroCuatro;
10     int limite = 100;
11     // Crea la semilla de los numeros aleatorios
12     srand(time(NULL));
13     // Genera dos numeros aleatorios
14     numeroUno = rand() % limite;
15     numeroDos = rand() % limite;
16     numeroTres = rand() % limite;
17     numeroCuatro = rand() % limite;
18     numeroFloatUno = (float)numeroUno + numeroTres / 100.0;
19     numeroFloatDos = (float)numeroDos + numeroCuatro / 100.0;
20     // imprime los cuatro numeros aleatorios enteros
21     printf("numeroUno    = %d\n", numeroUno);
22     printf("numeroDos     = %d\n", numeroDos);
23     printf("numeroTres    = %d\n", numeroTres);
24     printf("numeroCuatro = %d\n", numeroCuatro);
25     // imprime los dos numeros flotantes aleatorios
26     printf("numeroFloatUno = %f\n", numeroFloatUno);
27     printf("numeroFloatDos = %f\n", numeroFloatDos);
28     return 0;
29 }

```

Figura 2 Generador de numeros aleatorios.

Desarrollo

1. Realice un programa que se llame aleatoriocaracter.c para generar letras o caracteres aleatorios (dados los ejemplos de las figuras 1 y 2).
2. Del código mostrado en la figura 3, copielo y salvelo con el nombre del programa 1. Vea como trabaja y realice el mismo procedimiento para todos los tipos de datos.

Programa	Código fuente	Tipo contra todos los operadores
1	operadorconchar.c	char para cx y cy
2	operadorconshort.c	short para cx y cy
3	operadorconint.c	int para cx y cy
4	operadorconlongint.c	Long int para cx y cy
5	operadorconlong.c	long para cx, cy
6	operadorconlonglong.c	Long long para cx, cy
7	operadorconfloat.c	float cx y cy
8	operadorcondouble.c	double para cx y cy
9	Operadorconlongdouble.c	Long double para cv y cy

Incluya el codigo necesario que le de a las variables de entrada su valor aleatorio. Vea el ejemplo de la figura 3, recuerde que le falta código, no esta completo porque no muestra la salida de cada operación con cada operador, eso le toca a usted. Solo se muestra para que vea como se asignan los valores aleatorios a las variables con

las que se hará todo el proceso con cara uno de los operadores. Recuerde que sus variables deben estar declaradas al comienzo después de declarar la función principal y su llave de inicio.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define LIMITE 256

void imprimir(int entero) {
    printf("%d", entero);
}

int main(int argc, char *argv[]) {
    srand(time(NULL));
    // tipos de datos
    unsigned char cx = rand() % LIMITE; //90;
    unsigned char cy = rand() % LIMITE; //64;

    // variables de casteo o mutacion
    char c;
    short s;
    int i;
    long l;
    float f;
    double d;
    /***** CARACTERES char *****/
    printf("Caracter cx: %c %d\n", cx, (int)cx);
    printf("Caracter cy: %c %d\n", cy, (int)cy );

    // Operadores Aritmeticos:
    printf("%c * %c = ", cx, cy);
    cx = (char) (cx * cy);
    printf("%c %d\n", cx, (int)cx);

    printf("%c / %c = ", cx, cy);
    cx = (char) (cx / cy);
    printf("%c\n", cx);

    printf("%c mod %c = ", cx, cy);
    cx = (char) (cx % cy);
    printf("%c\n", cx);

    printf("%c + %c = ", cx, cy);
    cx = (char) (cx + cy);
    printf("%c\n", cx);

    printf("%c - %c = ", cx, cy);
    cx = (char) (cx - cy);
```

```

printf("%c\n", cx);

cx++;
printf("cx++ = %c\n", cx);
cx--;
printf("cx-- = %c\n", cx);
++cx;
printf("++cx = %c\n", cx);
--cx;
printf("--cx = %c\n", cx);
cx = (char) +cy;
printf("cx = +cy = %c\n", cx);
cx = (char) -cy;
printf("cx = -cy = %c\n", cx);
// Relacionales and logicos:
if(cx > cy) {
    printf("%c es mayor que %c\n", cx, cy);
}
if(cx >= cy) {
    printf("%c es mayor o igual que %c\n", cx, cy);
}
if(cx < cy) {
    printf("%c es menor que %c\n", cx, cy);
}
if(cx <= cy) {
    printf("%c es menor o igual que %c\n", cx, cy);
}
if(cx == cy) {
    printf("%c es igual a %c\n", cx, cy);
}
if(cx != cy) {
    printf("%c es diferente a %c\n", cx, cy);
}
//! imprimir(!x);
//! imprimir(x && y);
//! imprimir(x || y);
// operadores de Bits:
cx = (char) ~cy;
printf("cx = ~cy = %c\n", cx);

printf("%c & %c = ", cx, cy);
cx = (char) (cx & cy);
printf("%c\n", cx);

printf("%c | %c = ", cx, cy);
cx = (char) (cx | cy);
printf("%c\n", cx);

printf("%c ^ %c = ", cx, cy);
cx = (char) (cx ^ cy);
printf("%c\n", cx);

```

```

printf("%c << 1 = ", cx);
cx = (char) (cx << 1);
printf("%c\n", cx);

printf("%c >> 1 = ", cx);
cx = (char) (cx >> 1);
printf("%c\n", cx);

// Asignacion compuesta:
printf("%c += %c = ", cx, cy);
cx += cy;
printf("%c\n", cx);

printf("%c -= %c = ", cx, cy);
cx -= cy;
printf("%c\n", cx);

printf("%c *= %c = ", cx, cy);
cx *= cy;
printf("%c\n", cx);

printf("%c /= %c = ", cx, cy);
cx /= cy;
printf("%c\n", cx);

printf("%c mod= %c = ", cx, cy);
cx %= cy;
printf("%c\n", cx);

printf("%c <= 1 = ", cx);
cx <= 1;
printf("%c\n", cx);

printf("%c >= 1 = ", cx);
cx >= 1;
printf("%c\n", cx);

printf("%c &= %c = ", cx, cy);
cx &= cy;
printf("%c\n", cx);

printf("%c ^= %c = ", cx, cy);
cx ^= cy;
printf("%c\n", cx);

printf("%c |= %c = ", cx, cy);
cx |= cy;
printf("%c\n", cx);

// Casting o mutacion

```

```

s = (short) cx;
printf("%d\n", s);
i = (int) cx;
printf("%d\n", i);
l = (long) cx;
printf("%ld\n", l);
f = (float) cx;
printf("%f\n", f);
d = (double) cx;
printf("%lf\n", d);
return 0;
}

```

Figura 3 Código del programa operadorconint.c

3. Recuerde reportar al menos en el programa de caracteres cuando se suma, resta, divide, multiplica cuando se sale de rango por la izquierda o por la derecha cuantas vueltas dio y porque cayo en ese valor.
4. Emplee valores de creación de caracteres, numeros enteros y flotantes tan grandes como se pueda para salir del rango del tipo de dato y exponga sus conclusiones de lo que ocurre cuando se sale un tipo de su rango de acuerdo a la teoria.
5. Reporte sus conclusiones y sus referencias bibliográficas.